

Guida per Sviluppatori

Contribuire al Progetto, Ambiente di Sviluppo e Architettura del Codice

• Contribuire al Progetto, Ambiente di Sviluppo e Architettura del Codice

Categoria: Documentazione Tecnica

Prerequisito: Paper 02, Paper 05, Paper 12

• 1. Introduzione per Contributori

WhyCremisi è un progetto open-source (MIT) e accoglie contributi da sviluppatori audio, ingegneri del suono e appassionati di AI.

— 1.1 CODICE DI CONDOTTA

Tutti i contributori devono attenersi a un codice di condotta basato su rispetto, inclusività e collaborazione tecnica. Non tolleriamo discriminazioni, harassment o comportamenti antisociali.

— 1.2 COME INIZIARE

◆ OVERVIEW

1. Leggi Paper 01-07 per comprendere la visione
2. Configura l'ambiente di sviluppo (Sezione 2)
3. Cerca un issue labelato "good first issue"
4. Commenta l'issue per assegnartelo
5. Fork → Branch → Commit → PR

— 1.3 ISSUE TRACKER

Usiamo GitHub Issues. Label tassonomia:

LABEL	SIGNIFICATO
bug	Bug confermato
enhancement	Nuova funzionalità
good first issue	Adatto a nuovi contributori
help wanted	Richiesta assistenza
plugin-db	Riguarda il database plugin
daw-integration	Riguarda integrazione DAW
ui	Riguarda il frontend React
docs	Documentazione

— 1.4 FORUM SVILUPPATORI

[NOTE] Il canale #dev su Discord del progetto WhyCremisi è il luogo principale per discussioni tecniche, dubbi su architettura e code review informali. Tutte le decisioni tecniche rilevanti vengono documentate in GitHub Discussions.

- 2. Ambiente di Sviluppo

— 2.1 REQUISITI MINIMI

COMPONENTE	VERSIONE	NOTE
macOS	12.7+	Xcode 15.0+ richiesto
Windows	10 22H2+	VS 2022 17.8+
Linux	Ubuntu 22.04+	GCC 13+ o Clang 16+
JUCE	8.0.12	Submodule Git
CMake	3.28+	Build system
Node.js	18+	Frontend toolchain
npm	9+	Package manager

— 2.2 SETUP INIZIALE

```
# Clona il repository
git clone https://github.com/whycremisi/whycremisi.git
cd whycremisi

# Inizializza i submodule (JUICE, nlohmann-json, oscpack)
git submodule update --init --recursive

# Build C++ (macOS)
cmake -B build -G Xcode -DCMAKE_BUILD_TYPE=Debug
cmake --build build

# Build frontend
cd webview-ui
npm install
npm run dev      # sviluppo con HMR
```

— 2.3 IDE CONSIGLIATI

◆ OVERVIEW

Piattaforma IDE Configurazione

macOS Xcode 15+ cmake -G Xcode → .xcodeproj

Tutte VS Code Estensione CMake + clangd

Tutte CLion Apri CMakeLists.txt direttamente

Windows Visual Studio cmake -G "Visual Studio 17 2022"

— 2.4 FORMATTAZIONE AUTOMATICA

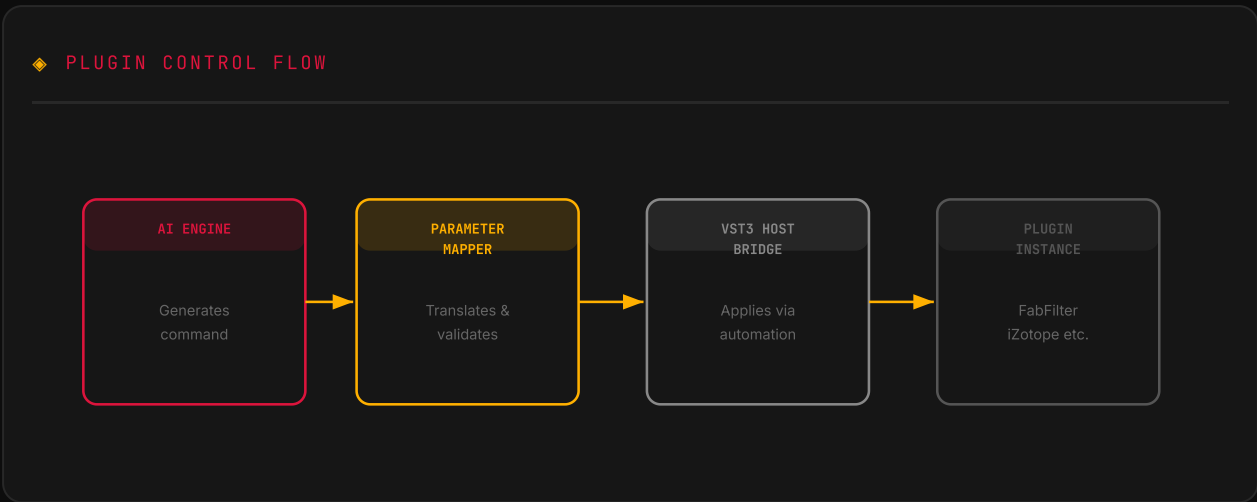
[NOTE] Il progetto usa `.editorconfig` e `.clang-format`. Assicurati che il tuo IDE li rispetti. Per VS Code, installa le estensioni `EditorConfig for VS Code` e `clangd`.

```
# .editorconfig (estratto)
root = true

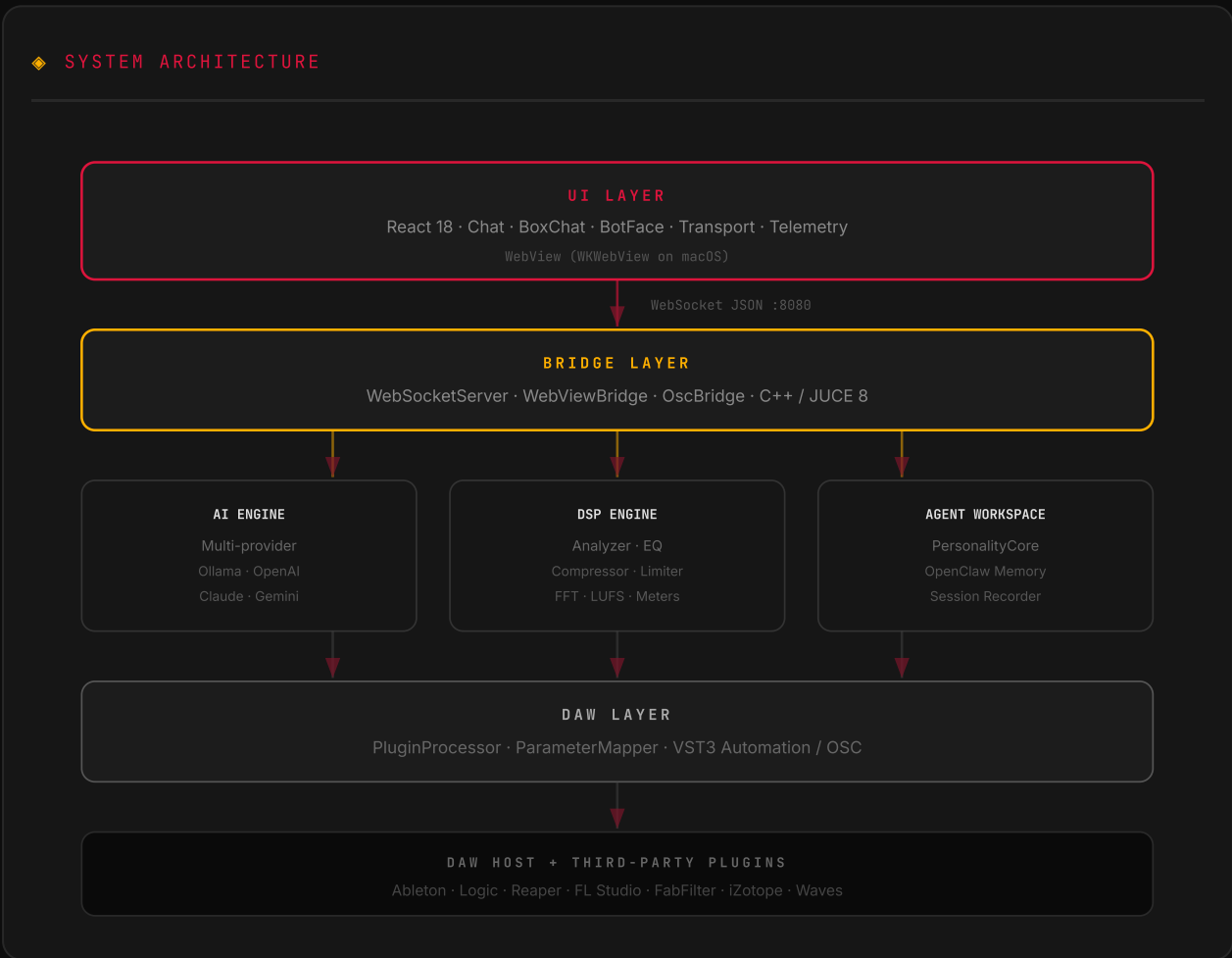
[*]
indent_style = space
indent_size = 4
end_of_line = lf
charset = utf-8
trim_trailing_whitespace = true
insert_final_newline = true

[*.{js,jsx,ts,tsx,css,json}]
indent_size = 2
```

— 3.1 STRUTTURA DELLE DIRECTORY



— 3.2 FLUSSO DATI



Legenda:

DIREZIONE	PROTOCOLLO	DATI
UI → C++	WebSocket JSON	<code>ai.prompt</code> , <code>daw.command</code> , <code>plugin.control</code>
C++ → UI	WebSocket JSON	<code>ai.stream</code> , <code>daw.transport</code> , <code>daw.meter</code> , <code>plugin.state</code>
C++ → DAW	OSC / UDP	<code>transport/play</code> , <code>track/volume</code> , <code>plugin/param</code>
DAW → C++	OSC / UDP	<code>transport/position</code> , <code>meter/level</code> , <code>track/info</code>
C++ → AI	HTTP (REST/SSE)	Prompt arricchito con contesto sessione
AI → C++	HTTP (SSE stream)	Risposta in chunk con tool calls

• 4. Convenzioni di Codice

— 4.1 C++ (JUICE / STANDARD)

ENTITÀ	CONVENTION	ESEMPIO
Classi	PascalCase	<code>class OscBridge</code>
Metodi	camelCase	<code>void processBlock()</code>
Variabili	camelCase	<code>int sampleRate</code>
Costanti	kPrefixedCamelCase	<code>constexpr int kMaxBufferSize</code>
Enum	PascalCase + k prefix	<code>enum class kState</code>
File	PascalCase	<code>AgentWorkspace.cpp</code>
Indentazione	4 spazi (no tab)	—
Namespace	lowercase	<code>namespace whycremisi</code>

— 4.2 REACT / JAVASCRIPT

ENTITÀ	CONVENTION	ESEMPIO
Componenti	PascalCase	<code>function EqBox()</code>
Funzioni	camelCase	<code>const handleStream = ()</code>
Variabili	camelCase	<code>const wsClient</code>
Costanti	UPPER_SNAKE	<code>const WS_PORT = 8080</code>
File componenti	PascalCase	<code>BotFace.jsx</code>
File utility	camelCase	<code>whycremisi-bridge.js</code>
Indentazione	2 spazi (no tab)	—

— 4.3 COMMENTI E DOCUMENTAZIONE

◆ OVERVIEW

Regola fondamentale:

- Commenti nel codice → INGLESE
- Documentazione pubblica → ITALIANO (per il mercato target)
- File header/copyright → INGLESE

```
// OK – commento in inglese per codice
void processBlock(AudioBuffer<float>& buffer, MidiBuffer& midi)
{
    // Apply forward gain compensation
    float makeupGain = MathUtil::gainToLinear(3.0f);
    buffer.applyGain(makeupGain);
}
```

```
// OK – commento in inglese per logica
function EqBox({ data }) {
    // Format frequency band data for display
    const bands = data.bands.map((b) => ({
        freq: `${b.frequency}Hz`,
        gain: b.gain.toFixed(1),
        q: b.q.toFixed(1),
    }));
}
```

— 4.4 STILE GENERALE

[NOTE] Tutto il codice C++ deve passare `clang-format` con il file `.clang-format` del progetto. Il codice React deve passare ESLint con la configurazione condivisa. Le PR che violano la formattazione vengono segnalate automaticamente dal CI.

• 5. Aggiungere un Nuovo Plugin al Database

— 5.1 PROCESSO

◆ DATA SCHEMA

1. Apri `plugins.json` in `docs/plugins/`
2. Aggiungi entry seguendo lo schema Paper 13
3. Compila tutti i campi: id, nome, produttore, parametri (con ID VST3/AU), preset, categorie
4. Esegui validazione
5. Crea PR con label "plugin-db"

— 5.2 ESEMPIO ENTRY

```
{
  "id": "fabfilter-pro-q-3",
  "name": "Pro-Q 3",
  "manufacturer": "FabFilter",
  "format": ["VST3", "AU", "AAX"],
  "categories": ["EQ", "Dynamic EQ"],
  "parameters": [
    {
      "id": "band1_freq",
      "name": "Band 1 Frequency",
      "vst3Id": 0,
      "range": { "min": 10, "max": 30000, "unit": "Hz" },
      "default": 1000
    },
    {
      "id": "band1_gain",
      "name": "Band 1 Gain",
      "vst3Id": 1,
      "range": { "min": -30, "max": 30, "unit": "dB" },
      "default": 0
    },
    {
      "id": "band1_q",
      "name": "Band 1 Q",
      "vst3Id": 2,
      "range": { "min": 0.1, "max": 100, "unit": "" },
      "default": 1.0
    }
  ],
  "presets": [
    { "name": "Vocal Presence", "params": { "band1_freq": 3000, "band1_gain": 2.5, "band1_q": 2.0 } }
  ]
}
```

— 5.3 VALIDAZIONE

```
node scripts/validate-plugin.js docs/plugins/fabfilter-pro-q-3.json
# Output atteso: ✅ Plugin fabfilter-pro-q-3 validato con successo
```

• 6. Aggiungere un Nuovo Comando DAW

— 6.1 PROCESSO

◆ TEST PIPELINE

1. Aggiungi metodo virtuale in `IDawHandler`
2. Implementa in `AbletonDawHandler` e `ReaperDawHandler`
3. Aggiungi tool definition in `Orchestrator`
4. Mappa comando OSC ↔ messaggio WebSocket
5. Test con DAW reale (Ableton o Reaper)

Step 1 — Interfaccia:

```
// src/daw/IDawHandler.h
class IDawHandler
{
public:
    virtual ~IDawHandler() = default;

    virtual void play() = 0;
    virtual void stop() = 0;
    virtual void setTrackVolume(int trackIndex, float volume) = 0;

    // ✨ Nuovo comando
    virtual void quantizeClip(int trackIndex, int clipIndex, float swing) = 0;
};
```

Step 2 — Implementazione Reaper:

```
// src/daw/ReaperDawHandler.cpp
void ReaperDawHandler::quantizeClip(int trackIndex, int clipIndex, float swing)
{
    // Reaper OSC API: /track/{idx}/item/{clipIdx}/quantize
    oscSender->sendMessage(
        osc::OutboundPacketStream(buffer, kBufferSize)
        << osc::BeginMessage("/track")
        << trackIndex
        << osc::BeginMessage("/item")
        << clipIndex
        << osc::BeginMessage("/quantize")
        << swing
        << osc::EndMessage
    );
}
```

Step 3 — Tool Definition:

```
// In AgentWorkspace::getTools()
Tool quantizeTool{
    .name = "daw_quantize_clip",
    .description = "Quantizza un clip MIDI sulla traccia specificata",
    .parameters = {
        {"trackIndex", Type::kInteger, "Indice traccia (0-based)"},
        {"clipIndex", Type::kInteger, "Indice clip (0-based)"},
        {"swing", Type::kFloat, "Swing amount (0.0-1.0)"}
    }
};
```

[NOTE] L'handler WebSocket deve tradurre il messaggio in arrivo (`{ type: "daw.command", payload: { command: "quantize_clip", ... } }`) nella chiamata al metodo corrispondente su `IDawHandler`. Il mapping vive in `OscBridge::handleCommand()`.

• 7. Aggiungere un Nuovo Box UI

— 7.1 PROCESSO

◆ TEST PIPELINE

1. Crea `webview-ui/src/boxes/NomeBox.jsx`
2. Segui pattern `BoxContext` + `motion` (Framer Motion)
3. Registra il nuovo box in `BoxChat.jsx`
4. Test con dati mock

— 7.2 ESEMPIO: METERBOX

```
// webview-ui/src/boxes/MeterBox.jsx
import { motion } from 'framer-motion';
import { useBoxContext } from '../hooks/useBoxContext';

export default function MeterBox({ data }) {
  const { onAction } = useBoxContext();

  return (
    <motion.div
      className="rounded-xl bg-zinc-800 p-4"
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
    >
      <h3 className="text-sm font-medium text-zinc-400">Level Meter</h3>
      <div className="mt-2 flex items-end gap-1">
        {data.levels.map((level, i) => (
          <div
            key={i}
            className="w-4 rounded-t bg-emerald-500 transition-all"
            style={{ height: `${level * 100}%` }}
          />
        ))}
      </div>
      <div className="mt-2 flex justify-between text-xs text-zinc-500">
        <span>L</span>
        <span>{data.peak.toFixed(1)} dB</span>
        <span>R</span>
      </div>
    </motion.div>
  );
}
```

— 7.3 REGISTRAZIONE IN BOXCHAT

```
// webview-ui/src/components/BoxChat.jsx
import MeterBox from '../boxes/MeterBox';

const BOX_RENDERERS = {
  eq:      EqBox,
  compressor: CompressorBox,
  level:    MeterBox,      // ← Nuovo box registrato
  spectrogram: SpectrogramBox,
};

export default function BoxChat({ messages }) {
  return messages.map((msg) => {
    const Renderer = BOX_RENDERERS[msg.type];
    return Renderer ? <Renderer key={msg.id} data={msg.data} /> : null;
  });
}
```

• 8. Test

— 8.1 TEST C++ (JUICE UNITTEST)

```
# Build e run test nativi
cmake --build build --target WhyCremisi_UnitTests
./build/WhyCremisi_UnitTests

# Output atteso:
# [OK] 32 test passed (4 test suites)
```

— 8.2 TEST FRONTEND (VITEST)

```
cd webview-ui
npm test

# Output atteso:
# PASS  src/__tests__/BoxChat.test.jsx
# PASS  src/__tests__/useWebSocket.test.js
```

— 8.3 LINT

```
cd webview-ui
npm run lint

# Nessun output = tutto pulito
# Errori ESLint bloccano il merge
```

[NOTE] Il CI esegue automaticamente test C++ e frontend su ogni PR. La build fallisce se i test non passano.

• 9. Pull Request Process

— 9.1 BRANCH NAMING

PREFIX	USO
<code>feature/</code>	Nuova funzionalità
<code>fix/</code>	Bug fix
<code>docs/</code>	Documentazione
<code>refactor/</code>	Refactoring
<code>test/</code>	Aggiunta/modifica test
<code>chore/</code>	Manutenzione (dipendenze, CI, build)

— 9.2 COMMIT MESSAGE CONVENTION

```
type(scope): description
```

Tipi: feat, fix, docs, refactor, test, chore

Esempi:

```
feat(daw): add quantize clip command
fix(ui): correct meter bar overflow on narrow screens
docs(plugin-db): add FabFilter Pro-R2 entry
refactor(bridge): extract OSC message parser
test(ai): add streaming timeout edge case
```

— 9.3 REVIEW E MERGE

◆ TEST PIPELINE

1. Fork del repository
2. Crea branch con naming corretto
3. Committa seguendo la convention
4. Apri PR con label appropriata
5. Almeno 1 approvazione da un maintainer
6. CI verde (test + lint + build)
7. Squash merge su main

• 10. Release Process

— 10.1 PRE-RELEASE CHECKLIST

- ❑ Tutti i test C++ passano
- ❑ Tutti i test frontend passano
- ❑ npm run lint senza errori
- ❑ Build su macOS, Windows e Linux verificata
- ❑ Documentazione aggiornata (Paper correlati)
- ❑ CHANGELOG.md aggiornato
- ❑ Version bump in CMakeLists.txt (segue Paper 12)

— 10.2 VERSION BUMP

◆ OVERVIEW

CMakeLists.txt

project(WhyCremisi VERSION 0.5.0) # → es: 0.4.0 → 0.5.0

— 10.3 TAG E GITHUB RELEASE

```
git tag -a v0.5.0 -m "v0.5.0 – DAW quantization, new plugin DB entries"
git push origin v0.5.0
```

[NOTE] La GitHub Release viene creata automaticamente da CI al push del tag, con asset precompilati per le tre piattaforme.

◆ TEST PIPELINE

Paper 15 / GUIDA SVILUPPATORI WhyCremisi

Prossimo: Paper 16 — Testing e QA